

SIMATS ENGINEERING
ASSESSMENT TOOL 2: CASE STUDY
COMPUTER VISION WITH OPENCV

Course Code: DSA02

Course Outcome Covered: CO3

Name	O.S.Sabiha
Register Number	192424264
Department	Artificial Intelligence and Data Science
Signature	sabiha

1. Preprocessing and Feature Detection

When images are captured under different lighting conditions, their brightness and contrast may change. This affects feature detection because feature detectors depend on intensity changes, gradients, corners, and local patterns. A feature that is clearly visible in one image may become weak or disappear in another image.

Preprocessing makes images more consistent before feature detection. Techniques such as grayscale conversion, histogram equalization, contrast enhancement, and noise reduction can be applied. Histogram equalization improves contrast, while Gaussian or median filtering reduces unwanted noise.

After preprocessing, important structures such as corners, edges, and textures become clearer. Therefore, feature detectors such as Harris, Shi-Tomasi, SIFT, or ORB can detect features more consistently.

Conclusion: Preprocessing improves feature detection by reducing lighting variations and noise while enhancing important image structures.

2. Edge vs Corner Detection

Edge detection identifies locations where image intensity changes significantly. It is useful for detecting boundaries and straight structures, but an edge usually does not provide a unique point for feature matching. Along a long straight edge, many points may look similar.

Corner detection identifies points where intensity changes significantly in two directions. Therefore, corners provide distinctive and localized keypoints that are easier to identify across different images.

For example, the corner of a window can be uniquely identified, whereas any point along the window's straight boundary may be ambiguous.

Algorithms such as Harris Corner Detector and Shi-Tomasi Corner Detector are suitable for finding such keypoints.

Conclusion: Edge detection is useful for boundaries, but corner detection is better for feature matching because corners are more distinctive and provide stable keypoints.

3. Hough Transform in Noisy Images

The Hough Transform detects geometric shapes by converting image points into a parameter space. In a noisy image, unwanted pixels may also vote in the parameter space. This can create false peaks and result in incorrect or multiple lines.

Noise should therefore be reduced before applying the Hough Transform. Gaussian filtering or median filtering can remove noise. An edge detector such as Canny can then be applied to obtain cleaner edges.

Hough parameters should also be selected carefully. Important parameters include the accumulator threshold, minimum line length, and maximum line gap. Increasing the threshold can reduce false detections, while appropriate line length and gap values can eliminate insignificant lines.

Conclusion: Noise affects Hough Transform by generating false votes. Filtering, accurate edge detection, and proper parameter tuning improve line detection accuracy.

4. Feature Matching under Transformations

Traditional feature matching may fail when the same object appears at different scales or orientations because the appearance and coordinates of its features change.

Scale-invariant methods are designed to identify the same local structures even when their size changes. SIFT (Scale-Invariant Feature Transform) is a common example. It detects features at multiple scales and creates descriptors that are relatively resistant to rotation and scale changes.

For example, if an object appears small in one image and large in another, SIFT can still identify corresponding features.

Conclusion: Scale- and rotation-invariant feature methods perform better because they describe local image structures independently of their exact size and orientation.

5. PCA for Feature Optimization

A computer vision system may extract hundreds or thousands of features. Many of these features can be redundant or highly correlated. Processing all of them increases memory usage and computational cost.

Principal Component Analysis (PCA) transforms the original features into a smaller set of new variables called principal components. The first components contain the largest amount of variance or information in the data.

For example, 100 features may be reduced to 20 principal components while retaining most of the important information.

The reduced feature representation requires less storage and makes algorithms faster.

Conclusion: PCA reduces dimensionality, removes redundancy, decreases computational cost, and improves the efficiency of feature processing while retaining the most important information.

6. Preprocessing Impact on Edge Detection

Noise and uneven illumination can cause an edge detector to identify unwanted intensity changes. As a result, real edges may become fragmented and false edges may appear.

Preprocessing can solve this problem. Gaussian filtering reduces noise, while contrast normalization or histogram equalization can improve illumination. Adaptive thresholding may also be useful when illumination varies across the image.

After preprocessing, the Canny edge detector can produce cleaner and more continuous edges.

General process: Input Image → Noise Reduction → Contrast/Illumination Correction → Canny Edge Detection → Clean Edges

Conclusion: Preprocessing improves edge detection by removing unwanted variations and making actual boundaries clearer and more continuous.

7. Corner Detection vs Edge Detection in Tracking

Edges represent boundaries, but they may not provide unique locations for tracking. A moving object's edge can shift, change shape, or become partially hidden between frames. This makes edge-based tracking less reliable.

Corners are more distinctive because they represent points where intensity changes in multiple directions. These points can be tracked from one frame to another.

Shi-Tomasi is commonly used for selecting strong corners, and optical flow methods such as Lucas-Kanade can track these points between frames.

For example, corners of a moving vehicle can be tracked more reliably than its long straight edges.

Conclusion: Corner points provide stable and distinctive locations, making them more suitable for object tracking across consecutive frames.

8. Hough Transform for Shape Detection

The Hough Transform can detect circles by identifying combinations of possible center coordinates and radius values. However, complex backgrounds may contain many circular or curved structures. These structures can create unwanted votes and false detections.

Preprocessing can improve the result by removing noise and unnecessary background information. Gaussian or median filtering can reduce noise, while Canny edge detection can provide cleaner input.

Parameters such as minimum radius, maximum radius, accumulator threshold, and minimum distance between circle centers should also be tuned.

If the threshold is too low, many false circles may be detected. If it is too high, real circles may be missed.

Conclusion: Hough Circle detection works better when the input is clean and the radius, threshold, and distance parameters are properly selected.

9. Feature Matching under Illumination Changes

Feature matching can fail when two images of the same scene have different lighting. Changes in brightness and contrast alter pixel intensities and may change the appearance of local features.

Preprocessing can reduce this problem through grayscale conversion, histogram equalization, contrast normalization, and illumination correction.

Robust descriptors can also improve matching. Methods such as SIFT and ORB describe local structures rather than depending only on exact pixel intensity values.

A suitable matching method, such as Brute-Force matching with an appropriate distance metric or FLANN, can then be used.

Conclusion: Illumination normalization combined with robust feature descriptors improves the reliability of feature matching.

10. PCA in Face Recognition

Face images contain a very large number of pixels. For example, a 100×100 grayscale image contains 10,000 pixel values. Processing thousands of dimensions for every face can be computationally expensive.

PCA reduces these dimensions by identifying the directions containing the largest amount of variation. In face recognition, the resulting principal components are often called Eigenfaces.

Instead of comparing all original pixels, the system can compare a smaller number of PCA features.

Example: 10,000 pixel features $\rightarrow$ PCA $\rightarrow$ 100 important components

This reduces computation while retaining major facial characteristics.

Conclusion: PCA makes face recognition faster and more efficient by reducing high-dimensional image data while preserving important facial information.

11. Edge Detection in Low Contrast Images

In a low-contrast image, the intensity difference between an object and its background is small. Therefore, edge detectors may not identify the boundaries correctly.

Preprocessing can improve the contrast before edge detection. Histogram equalization, CLAHE (Contrast Limited Adaptive Histogram Equalization), and contrast stretching are useful techniques.

CLAHE is particularly useful when different areas of the image have different contrast levels because it improves local contrast.

After enhancement, algorithms such as Canny can identify stronger and clearer edges.

Conclusion: Contrast enhancement increases the intensity difference between important regions, allowing edge detection algorithms to identify boundaries more effectively.

12. Corner Detection in Textured Regions

Highly textured regions contain many intensity changes in different directions. A corner detector may therefore identify a very large number of keypoints.

Although having many keypoints may seem useful, excessive keypoints increase computational cost and may include weak or redundant features. This can also make matching less efficient.

Feature selection can be improved by using a response threshold to retain only strong corners. Non-maximum suppression can remove nearby redundant points. A maximum number of keypoints can also be specified.

Another approach is to use quality thresholds and minimum distance constraints between selected points.

Conclusion: Strong and well-distributed corners should be selected instead of keeping every detected keypoint. This improves both efficiency and matching quality.

13. Feature Matching with Noise

Noise introduces unwanted intensity variations into an image. Feature detectors may interpret these variations as real features, resulting in incorrect keypoints and unreliable descriptors.

When descriptors are generated from noisy regions, the descriptors of corresponding features may become different. Therefore, matching accuracy decreases.

Preprocessing should be performed before feature detection. Gaussian filtering is useful for reducing general noise, while median filtering is effective for salt-and-pepper noise.

After noise reduction, feature detection and descriptor extraction can be performed again.

Conclusion: Noise reduces feature matching reliability by creating false features and modifying descriptors. Proper filtering improves the quality of detected features and matching accuracy.

14. PCA and Information Loss

PCA reduces the number of dimensions, but aggressive reduction can remove useful information. If too few principal components are selected, important features may be discarded.

There is therefore a trade-off:

More components → More information retained but higher computation.

Fewer components → Faster processing but greater information loss.

The number of components should be selected based on the explained variance ratio. For example, components that collectively retain around 95% of the variance may provide a reasonable balance between information preservation and dimensionality reduction.

The optimal number also depends on the application and validation performance.

Conclusion: PCA should not be applied too aggressively. The number of components should be selected by considering explained variance, computational requirements, and the accuracy of the final application.

15. Combined Feature Detection and Matching

Feature detection and feature description are different stages. A detector identifies important keypoints, while a descriptor represents the appearance around those keypoints.

A system may correctly detect features but still fail during matching if the selected descriptor is unsuitable for the image conditions.

For example, SIFT descriptors are highly effective for scale and rotation changes, while ORB provides faster computation and is suitable for many real-time applications.

The descriptor must also be paired with an appropriate matching distance. Hamming distance is suitable for binary descriptors such as ORB, whereas Euclidean/L2 distance is commonly used with floating-point descriptors such as SIFT.

Therefore, descriptor selection should consider scale changes, rotation, illumination, noise, computational requirements, and the matching method.

Conclusion: Correct feature detection alone is not sufficient. Choosing an appropriate descriptor and compatible matching method is essential for reliable image matching.

Code of Conduct

I ___O.S.Sabiha(192424264)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ___sabiha_____